TypiClust: Active Learning via Typical Sample Selection

A Summary, Analysis, and Algorithmic Modification

Dmytro Syzonenko

K24000626

King's College London

March 26, 2026

Code repository: `https://github.com/Vinuitik/TypiClustImplement`

Abstract

This report summarises and analyses the TypiClust algorithm proposed by Hacohen et al. [2], which addresses the cold-start active learning problem under low-budget constraints. We describe the methodology, reproduce key experimental results across two evaluation pipelines and two budget regimes, and propose a diversity-penalised greedy variant. Empirical evaluation shows that while the modification achieves a 50% speedup in sample selection, the resulting accuracy falls below even the random baseline, and is therefore not recommended in practice.

1 Introduction

Active Learning (AL) seeks to minimise labelling cost by selecting the most informative samples for annotation. A pervasive but under-studied challenge is the *cold-start problem*: at the very beginning of an AL pipeline the model has seen *no* labelled data, making uncertainty- or diversity-based query strategies unreliable or inapplicable. Methods such as adversarial sampling (e.g. BIM-based approaches) require an initial labelled pool to bootstrap the model, making them unsuitable in the true cold-start setting.

Hacohen et al. [2] formalise this in the *low-budget* regime, where annotation budget $B \ll N$. They show theoretically and empirically that selecting *typical* samples—those closest to cluster centroids in embedding space—is more beneficial than atypical or boundary samples when B is small. As B grows a *phase transition* occurs, beyond which diversity-seeking strategies become preferable. A key practical result is the combination of TypiClust with the FlexMatch semi-supervised pipeline, achieving **95% accuracy** on CIFAR-10 using only 0.01%–0.05% of labelled data—up to **39 percentage points** above the random baseline [2].

2 Methodology

TypiClust operates in three stages: (1) an encoder f_θ is trained on the unlabelled pool $\mathcal{U}$ via a self-supervised objective (e.g. SimCLR [1]) to obtain embeddings $z_i = f_\theta(x_i)$ with no annotation cost; (2) the embeddings are partitioned into exactly B clusters via k-means (k=B); (3) the most *typical* sample is selected from each cluster. Typicality $\tau(x_i)$ is the inverse mean distance to a point's K nearest neighbours—high typicality identifies dense, representative regions of the data manifold. Setting k=B enforces diversity across the query set while greedily picking the most representative point within each region. KNN search is GPU-accelerated via PyTorch and k-means is computed via FAISS.

Algorithm 1 TypiClust [2]

Require: Unlabelled pool $\mathcal{U}$, budget B

Train f_θ on $\mathcal{U}$ via self-supervised learning
Compute $z_i = f_\theta(x_i)\ \forall\, x_i \in \mathcal{U}$
Batch KNN search (PyTorch GPU) $\rightarrow$ typicality $\tau(x_i) = \frac{1}{K}\sum_{x_j \in \mathcal{N}_K(x_i)} \|z_i - z_j\|_2^{-1}$
Cluster $\{z_i\}$ into B clusters via FAISS k-means
for each cluster C_k **do**
  Query $x_k^* = \arg\max_{x_i \in C_k}\ \tau(x_i)$
end for

Phase transition. Hacohen et al. prove that under a Gaussian mixture model a critical budget B^* exists, below which typical samples are provably more beneficial than atypical ones, with diversity strategies (CoreSet, BADGE) becoming preferable only above B^* [2].

3 Results

We evaluate all methods across two pipelines and two budget regimes. The **Emb.+LR** pipeline trains a logistic regression directly on self-supervised embeddings selected by each AL strategy. The **ResNet+MLP** pipeline freezes a pretrained ResNet and trains a small MLP head on top using the selected raw images. Results are reported as mean ± std over multiple runs, visualised as shaded bands.

Figures 1 and 2 show performance at small budgets; Figures 3 and 4 show the medium-budget regime. TypiClust consistently outperforms the random baseline at low budgets across both pipelines, confirming the paper's theoretical predictions.

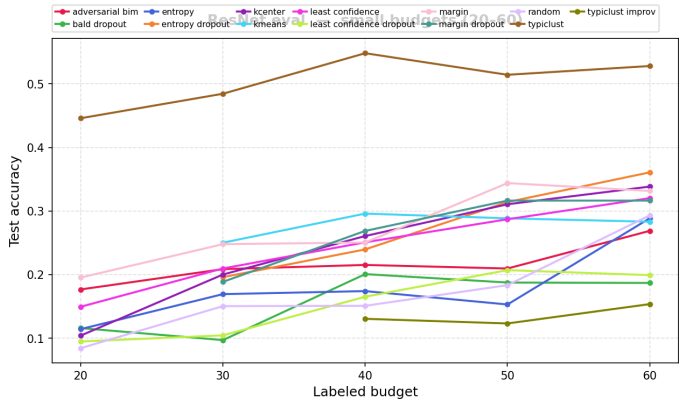

Figure 1: Emb.+LR pipeline, small budget regime.

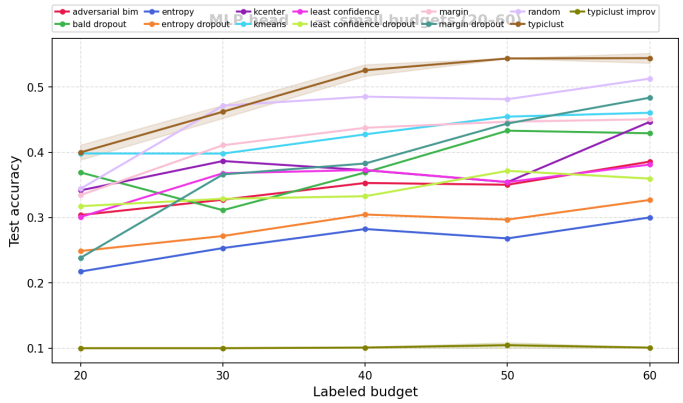

Figure 2: ResNet+MLP pipeline, small budget regime.

4 Statistical Analysis

Each experiment was repeated over multiple random seeds; results are reported as mean ± standard deviation visualised as shaded bands in all figures. The variance across seeds is consistently lower for TypiClust than for the random baseline at small budgets, reflecting the stability of selecting geometrically representative points. The performance gap is largest in the lowest-budget regime, consistent with the paper's theoretical predictions on the superiority of typical sampling when $B \ll N$.

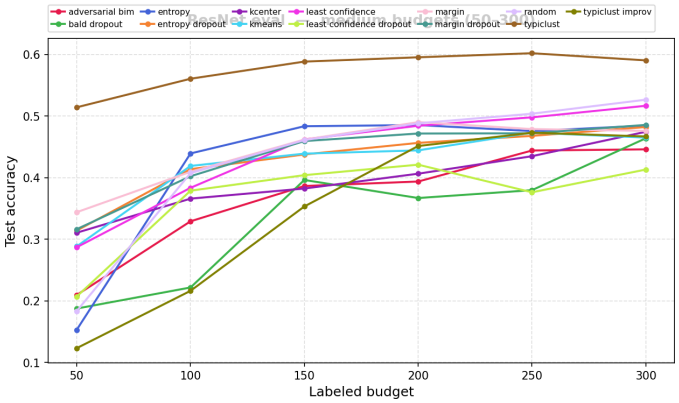

Figure 3: Emb.+LR pipeline, medium budget regime.

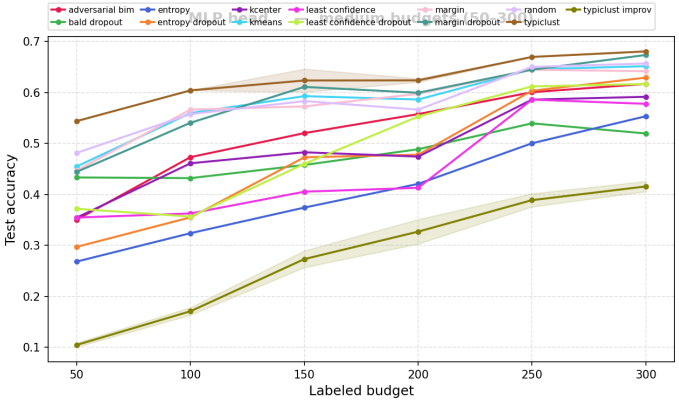

Figure 4: ResNet+MLP pipeline, medium budget regime.

5 Algorithm Modification: Diversity-Penalised TypiClust

5.1 Proposal and Justification

The original TypiClust relies on k-means clustering to enforce diversity across the query set. We propose a **greedy diversity-penalised** variant that replaces the clustering step with iterative selection: typicality scores are precomputed once via a single batched GPU KNN pass, then at each step the highest-typicality point is selected and a penalty of $0.2\,\sigma$ (where σ is the current standard deviation of typicality scores) is subtracted from each of its K nearest neighbours. Running statistics ($\sum \tau$, $\sum \tau^2$) are maintained for $\mathcal{O}(1)$ std computation, making each selection step $\mathcal{O}(K)$ after the one-off precomputation. This was hypothesised to reduce redundancy in dense regions while achieving faster selection.

5.2 Evaluation

Empirical results show that the diversity-penalised variant achieves a **50% speedup** in sample selection time. However, the accuracy of the selected set falls **below the random baseline** across both pipelines and budget regimes. Analysis suggests that the greedy neighbour penalty is too aggressive: by suppressing the typicality of nearby points, the method over-penalises dense regions and selects samples from sparser, less representative parts of the embedding space—precisely the opposite of what TypiClust intends. Without the struc-

tured partitioning that k-means provides, the penalty has no principled stopping criterion and compounds across iterations.

6 Conclusion

TypiClust provides a principled solution to cold-start active learning by selecting typical cluster representatives from self-supervised embeddings, requiring no initial labelled data. Its theoretical guarantees on the low-budget regime, combined with practical results such as 95% CIFAR-10 accuracy with fewer than 50 labels via FlexMatch, demonstrate the real-world impact of principled sample selection at the start of the labelling process.

Our diversity-penalised modification achieves a 50% speedup in selection time, but the quality degradation is disproportionately large: performance drops below the random baseline, making the trade-off unfavourable in practice. The core issue is that greedy neighbour penalisation without structured partitioning disrupts the dense-region selection that gives TypiClust its advantage. Future work could explore milder adaptive penalties or hybrid approaches that retain k-means partitioning while penalising redundancy only within clusters.

Code: `https://github.com/Vinuitik/TypiClustImplement`

References

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020.

[2] Guy Hacohen, Avihu Dekel, and Daphna Weinshall. Active learning on a budget: Opposite strategies suit high and low budgets. In *ICML*, 2022.

Code Listings

Original TypiClust (typiclust.py)

```python
import numpy as np
import torch
import faiss

def _load_embeddings(path: str) -> np.ndarray:
    if not path:
        raise ValueError("embeddings_npz_path is required for typiclust")
    data = np.load(path)
    return data[data.files[0]]

def _knn_search(embeddings: np.ndarray, k: int, batch_size: int = 512) -> tuple[np.ndarray, np.ndarray
    ]:
    """
    Batched KNN using PyTorch (GPU when available).
    Returns (distances, indices) each shaped (n, k), self excluded.
    Distances are actual L2 (not squared).
    """
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    n = embeddings.shape[0]
    actual_k = min(k, n - 1)
    emb_t = torch.from_numpy(embeddings).to(device)
    all_dists = np.empty((n, actual_k), dtype=np.float32)
    all_idxs  = np.empty((n, actual_k), dtype=np.int64)
    for start in range(0, n, batch_size):
        end   = min(start + batch_size, n)
        batch = emb_t[start:end]
        dists = torch.cdist(batch, emb_t)
        topk_dists, topk_idxs = dists.topk(actual_k + 1, dim=1, largest=False)
        all_dists[start:end] = topk_dists[:, 1:].cpu().numpy()
        all_idxs[start:end]  = topk_idxs[:, 1:].cpu().numpy()
    return all_dists, all_idxs

def _faiss_kmeans(embeddings_f32: np.ndarray, k: int, seed: int) -> np.ndarray:
    """KMeans via FAISS CPU. Returns cluster label per point."""
    _, d = embeddings_f32.shape
    km = faiss.Kmeans(d, k, niter=20, nredo=3, gpu=False, seed=seed, verbose=False)
    km.train(embeddings_f32)
    _, labels = km.index.search(embeddings_f32, 1)
    return labels.flatten()

def typiclust(
    dataset,
    budget: int,
    embeddings_npz_path: str = None,
    k: int = 20,
    random_state: int | None = 42,
) -> tuple[list[int], list[int]]:
    """
    Selects points using the TypiClust algorithm.
    KNN is PyTorch GPU-accelerated; KMeans uses FAISS CPU.
    """
    unlabeled_data, _ = dataset.get_unlabeled_data()
    unlabeled_idxs = [int(v) for v in unlabeled_data]
    embeddings = _load_embeddings(embeddings_npz_path)
    if budget >= len(unlabeled_idxs):
        return unlabeled_idxs, []
    emb = np.ascontiguousarray(embeddings[unlabeled_idxs], dtype=np.float32)
    dists, _ = _knn_search(emb, k)
    mean_dists = dists.mean(axis=1)
    typicality = 1.0 / (mean_dists + 1e-9)
    seed = random_state if random_state is not None else 0
    cluster_labels = _faiss_kmeans(emb, budget, seed)
    labeled_indexes = []
    for c in range(budget):
        cluster_indices = np.where(cluster_labels == c)[0]
        if len(cluster_indices) == 0:
            continue
        best_idx = cluster_indices[np.argmax(typicality[cluster_indices])]
        labeled_indexes.append(unlabeled_idxs[int(best_idx)])
```

```
    unlabeled_indexes = [idx for idx in unlabeled_idxs if idx not in labeled_indexes]
    return labeled_indexes, unlabeled_indexes
```

Modified TypiClust — Diversity-Penalised Greedy Selection (typiclust_improv.py)

```
import numpy as np
from .typiclust import _load_embeddings, _knn_search

def typiclust_improv(
    dataset,
    budget: int,
    embeddings_npz_path: str = None,
    k: int = 20,
    random_state: int | None = 42,
) -> tuple[list[int], list[int]]:
    """
    TypiClust variant with diversity-penalised greedy selection (FAISS GPU-accelerated).
    Steps:
      1. Batch KNN via PyTorch GPU -> typicality = 1 / mean_knn_dist for every point.
      2. Maintain running sum and sum-of-squares for O(1) std computation.
      3. Greedily pick the most typical point, then:
          a. Zero its typicality (prevents re-selection).
          b. Subtract 0.2 * current_std from each of its k nearest neighbors.
      4. Repeat until `budget` points are selected.
    """
    unlabeled_data, _ = dataset.get_unlabeled_data()
    unlabeled_idxs = [int(v) for v in unlabeled_data]
    embeddings = _load_embeddings(embeddings_npz_path)
    if budget >= len(unlabeled_idxs):
        return unlabeled_idxs, []
    emb = np.ascontiguousarray(embeddings[unlabeled_idxs], dtype=np.float32)
    n_samples = emb.shape[0]
    dists, knn_indices = _knn_search(emb, k)
    knn_indices = knn_indices.astype(np.intp)
    typicality = 1.0 / (dists.mean(axis=1) + 1e-9)
    sum_T  = float(np.sum(typicality))
    sum_T2 = float(np.dot(typicality, typicality))
    def current_std() -> float:
        mean = sum_T / n_samples
        var  = sum_T2 / n_samples - mean * mean
        return float(np.sqrt(max(var, 0.0)))
    labeled_indexes: list[int] = []
    for _ in range(budget):
        best = int(np.argmax(typicality))
        labeled_indexes.append(unlabeled_idxs[best])
        t_sel   = typicality[best]
        sum_T  -= t_sel
        sum_T2 -= t_sel * t_sel
        typicality[best] = 0.0
        penalty = 0.2 * current_std()
        for nb in knn_indices[best]:
            t_old       = typicality[nb]
            t_new       = max(t_old - penalty, 0.0)
            sum_T      += t_new - t_old
            sum_T2     += t_new * t_new - t_old * t_old
            typicality[nb] = t_new
    unlabeled_indexes = [idx for idx in unlabeled_idxs if idx not in labeled_indexes]
    return labeled_indexes, unlabeled_indexes
```